Asynchronous FIFO — CDC Design & UVM Verification

Gray-Coded Pointers, Reset-Aware Scoreboarding, and a Bug-Injection Regression Harness

Sasha Katne

PSU ECE 593 — Functional Verification (Team 3)

github.com/sashakatne/Team3_AsyncFIFO_S24_ECE593

The Problem

An **asynchronous FIFO** buffers data between two clock domains that share **no phase relationship**:

- **Write domain** — 80 MHz (12.5 ns period), bursty producer
- **Read domain** — 50 MHz (20 ns period), slower consumer
- 64 entries × 8 bits, no upstream backpressure

A correct implementation must guarantee:

- **Lossless transfer** — every accepted write is read back exactly once
- **Order preservation** — FIFO discipline across the clock boundary
- **Overflow protection** — writes blocked when `wFull`
- **Underflow protection** — reads blocked when `rEmpty`
- **No metastability** — pointer transfers tolerate the unrelated clock edges
- **Immediate flag behavior** — `wHalfFull` / `rHalfEmpty` assert on the threshold transaction, not the cycle after

Simulation with a single clock cannot exercise these obligations. Two unrelated clocks force every CDC hazard to actually occur.

Why an Asynchronous FIFO

Alternative	Why it loses for streaming CDC
Two-flop synchronizer on bare data	Only safe for single-bit signals; multi-bit data corrupts on inconsistent sampling
Bus handshake (`req` / `ack`)	Backpressure per word — kills throughput for streaming
Mailbox / single register	No queueing — slow consumer drops samples
Shared occupancy counter	The counter itself is an unsafe shared CDC state
Asynchronous FIFO (this work)	Per-domain pointers, Gray-coded across, queue-depth elasticity

A FIFO turns a CDC problem into a **pointer-synchronization problem** — a problem with a known, provably-safe solution.

CDC Hazards Being Defended Against

- **Metastability** — a flip-flop sampled within its setup/hold window resolves to an indeterminate value; resolution time is exponentially distributed
- **Multi-bit pointer sampling** — a raw binary increment can change multiple bits in the same cycle (`0111 → 1000`). Different bits cross the synchronizer in different cycles → spurious pointer values
- **Unsafe shared occupancy** — a `fifo_count` register written by both clocks has no safe sampling window
- **Reset asymmetry** — write and read domains are reset independently; a naïve scoreboard sees mid-flush state

> **Design principle:** every cross-domain signal must be either a single bit, or a Gray-coded pointer where exactly one bit changes per increment.

System Architecture — Five Functional Blocks

```
asynchronous_fifo (top, Post_M5/UVM/async_fifo.sv)
├── fifo_memory      storage array; gated write/read access
├── write_pointer    binary -> Gray wptr; wFull; wHalfFull
├── read_pointer     binary -> Gray rptr; rEmpty; rHalfEmpty
├── sync_w2r         wptr synchronized into the read clock domain
└── sync_r2w         rptr synchronized into the write clock domain
```

- One module per CDC concern — the synchronizers are **identical reusable blocks** parameterized by stage count and width
- Status flags are produced inside the *consuming* domain so consumers never need to cross-clock the result
- All pointers are **7 bits wide** (`ADDR_SIZE+1 = 6+1`) — the extra MSB is structural, not a debug aid

Design Decision #1 — Binary Locally, Gray Across

Pointer arithmetic stays in **binary** inside each domain (increment, occupancy diff, threshold compare are all trivial). Only the **cross-domain copy** is Gray.

```
// write_pointer module in async_fifo.sv — ports: clk=wclk, rst_n=wrst, inc=winc
assign binary_wptr_next = binary_wptr + (inc & ~wFull);              // local: binary
assign gray_wptr_next   = (binary_wptr_next>>1) ^ binary_wptr_next;  // cross: Gray

always_ff @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    wptr <= '0; binary_wptr <= '0; wFull <= '0; wHalfFull <= '0;
  end else begin
    wptr        <= gray_wptr_next;       // Gray pointer registered for cross-domain
    binary_wptr <= binary_wptr_next;     // local binary for arithmetic
    wFull       <= full_next;
    wHalfFull   <= half_full_next;
  end
end
```

Why: Gray code guarantees exactly **one bit transition per pointer increment**. Even if the destination domain samples mid-transition, the worst case is reading the old value or the new value — never a corrupted intermediate.

Design Decision #2 — One Extra Pointer Bit

The address space is 6 bits (64 entries), but the pointer is **7 bits**. The MSB encodes "lap parity" — how many times the pointer has wrapped.

Write ptr MSB	Read ptr MSB	Address bits equal?	State
`0`	`0`	yes	**empty**
`1`	`1`	yes	**empty** (both wrapped same number of times)
`0`	`1`	yes	**full** (writer one lap behind reader's perspective)
`1`	`0`	yes	**full**

```
// wrap-aware full detection — verbatim from write_pointer in async_fifo.sv
assign full_next = ((gray_wptr_next[ADDR_SIZE-2:0] == wq2_rptr[ADDR_SIZE-2:0]) &&
                    (gray_wptr_next[ADDR_SIZE-1:0] != wq2_rptr[ADDR_SIZE-1:0]) &&
                    (gray_wptr_next[ADDR_SIZE]     != wq2_rptr[ADDR_SIZE]));
```

The three conditions, on the **Gray-coded** pointers, jointly express "lower bits match AND MSB-pair differs" — the Gray-domain equivalent of "same address, opposite lap."

> Replacing this with `(gray_wptr_next == wq2_rptr)` is exactly `WPTR_FULLFLAG_BUG` — a real defect the verification environment is required to catch.

Design Decision #3 — Destination-Domain Synchronizers

A two-flop synchronizer must be clocked by the **consumer** of the synchronized signal, not the producer.

```
// async_fifo.sv — verbatim instantiations from the asynchronous_fifo top
sync #(.ADDR_SIZE(ADDR_SIZE)) sync_w2r (        // wptr → read domain: clocked in rclk
    .clk(rclk), .rst_n(rrst), .wData(wptr), .rData(rq2_wptr));

sync #(.ADDR_SIZE(ADDR_SIZE)) sync_r2w (        // rptr → write domain: clocked in wclk
    .clk(wclk), .rst_n(wrst), .wData(rptr), .rData(wq2_rptr));
```

The `sync` module reuses `.wData` / `.rData` as its own port names — they are the synchronizer's input and output, **not** the FIFO's data path. The actual signals being synchronized are the 7-bit Gray pointers.

The subtle trap: if the synchronizer were clocked in the *source* domain, the second flop would still be in the source domain — the destination logic would sample an unsynchronized value and the metastability tolerance evaporates.

This is also where `SYNC_BUG` lives — removing the second flop reduces the MTBF to single-flop levels and the scoreboard must detect the resulting mismatches.

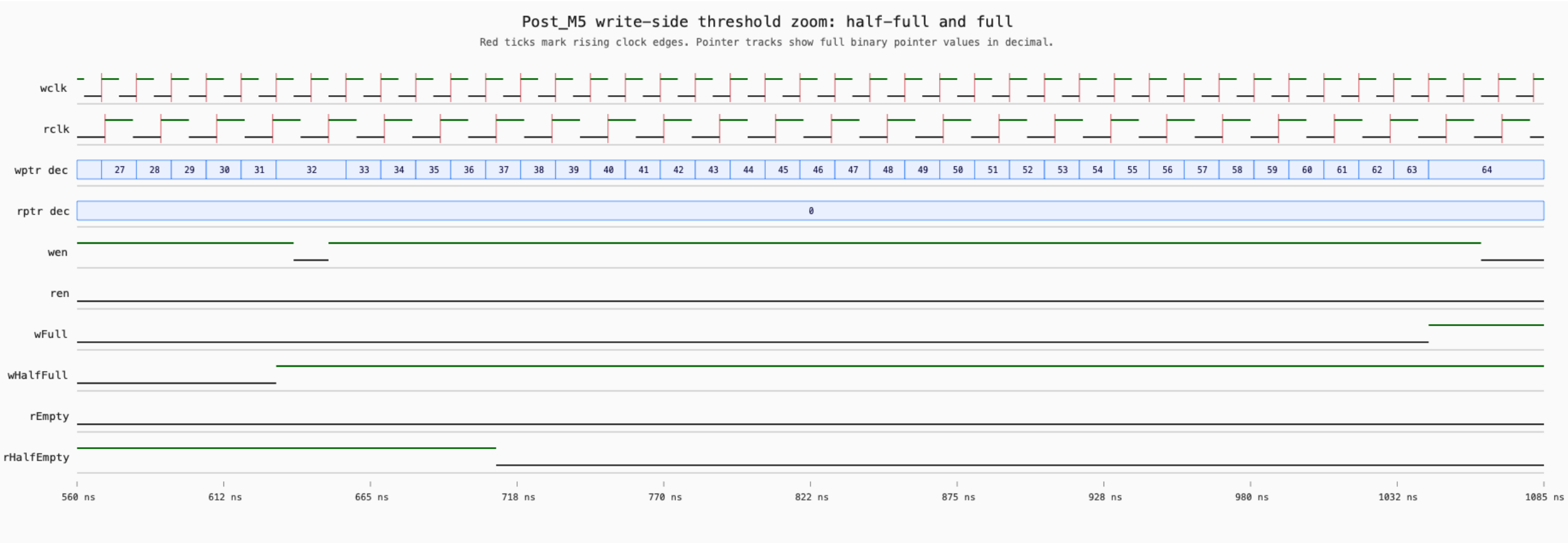
Design Decision #4 — Next-Pointer Half-Flag Timing

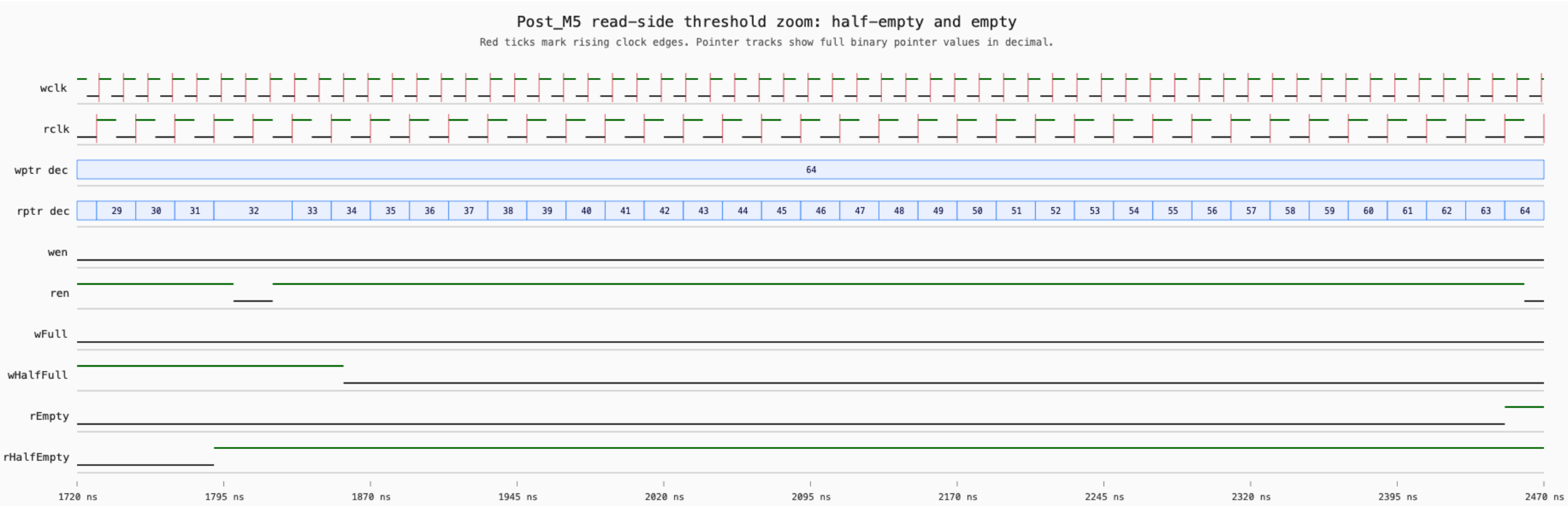
Half-full and half-empty are computed from **next-cycle local pointer** vs. **synchronized remote pointer** — never from a dual-clock occupancy counter.

```
// write_pointer.sv — half-full driven by NEXT local wptr (not current)
assign wptr_diff      = binary_wptr_next - binary_rptr_sync;
assign half_full_next = (wptr_diff >= 2**(ADDR_SIZE-1));   // 2**5 = 32 for depth 64

// read_pointer.sv — mirror on the read side
assign rptr_diff       = binary_wptr_sync - binary_rptr_next;
assign half_empty_next = (rptr_diff <= 2**(ADDR_SIZE-1));

// wHalfFull / rHalfEmpty are then registered from *_next inside the always_ff block.
```

Why this matters: with the *current* pointer the flag asserts **one cycle late** — the threshold transaction has already been accepted before the flag tells you it crossed. Using `*_next` makes the assertion edge align with the accepted transfer.

> M3 directed runs exposed the one-cycle-late behavior. Post_M5 fixes it. See Engineering Insight #1.


Post_M5 write-side threshold zoom: half-full and full
Red ticks mark rising clock edges. Pointer tracks show full binary pointer values in decimal.
wclk
rclk
wptr dec
27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64
rptr dec
0
wen
ren
wFull
wHalfFull
rEmpty
rHalfEmpty
560 ns
612 ns
665 ns
718 ns
770 ns
822 ns
875 ns
928 ns
980 ns
1032 ns
1085 ns


Post_M5 read-side threshold zoom: half-empty and empty
Red ticks mark rising clock edges. Pointer tracks show full binary pointer values in decimal.
wclk
rclk
wptr dec
64
rptr dec
29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64
wen
ren
wFull
wHalfFull
rEmpty
rHalfEmpty
1720 ns
1795 ns
1870 ns
1945 ns
2020 ns
2095 ns
2170 ns
2245 ns
2320 ns
2395 ns
2470 ns

UVM Testbench Architecture

```
tb_top  (Post_M5/UVM/async_fifo_top.sv)
├── DUT: asynchronous_fifo
├── intf: shared SystemVerilog interface (virtual intf via uvm_config_db)
└── uvm_test_top: fifo_random_test (default) | fifo_base_test (+define+BASE_TEST)
    └── fifo_env
        ├── write_agent
        │   ├── write_sequencer  ── drives constrained-random wData & winc
        │   ├── write_driver     ── pin-level wclk-domain stimulus
        │   └── write_monitor    ── publishes accepted writes (winc && !wFull)
        ├── read_agent
        │   ├── read_sequencer   ── drives rinc with controllable rate
        │   ├── read_driver      ── pin-level rclk-domain stimulus
        │   └── read_monitor     ── publishes accepted reads (rinc && !rEmpty)
        └── fifo_scoreboard      ── reset-aware queue reference model
```

- Two **independent** agents — write/read run on unrelated clocks, both publish to the scoreboard via analysis ports
- The interface is bound to both DUT and TB; **no DUT internals are inspected** by checkers

Race-Free Driver Discipline

The DUT samples `winc` / `rinc` / `wData` on the **rising clock edge**. Drivers therefore stimulate on the **falling edge** so signals are stable before the sampling edge.

```
// async_fifo_driver.sv — verbatim drive_write task
task drive_write (transaction_write txw);
    @(negedge intf_vi.wclk);                // signals settle a half-cycle before posedge
    this.intf_vi.winc  = txw.winc;
    this.intf_vi.wData = txw.wData;
endtask                                     // returns; DUT samples on next posedge
```

Why: if the driver wrote inside `@(posedge wclk)` instead, two `always @(posedge wclk)` blocks (the driver's and the DUT's) would race in the simulator scheduler. The DUT could sample either the previous or the new value depending on scheduler order — pass/fail becomes a non-deterministic artifact of the simulator. Driving at `negedge` puts a full half-cycle of settling between the change and the sample, eliminating the race and matching how synchronous logic actually receives signals on a board.

Accepted-Transfer Monitoring

The monitors publish **only accepted transfers**, not every stimulus pulse.

```
// async_fifo_write_monitor.sv — mon_write task (verbatim essentials)
@(negedge vif.wclk); #1;                   // sample just after the negedge
accepted = (vif.wrst === 1'b1) &&          // not in reset
           (vif.winc === 1'b1) &&          // write attempted
           (vif.wFull === 1'b0);           // and FIFO had room
sampled_wData = vif.wData;
@(posedge vif.wclk); #1;                   // wait through the DUT's sample edge
if (accepted) begin
    txw = transaction_write::type_id::create("txw");
    txw.wData = sampled_wData;
    port_write.write(txw);                 // → scoreboard analysis port
end
```

Why this distinction matters:

- The test deliberately drives `winc=1` while `wFull=1` to verify **overflow protection** — those writes must *not* enter the scoreboard reference queue, or the comparison desynchronizes
- Same logic on the read side for `rinc && !rEmpty` — underflow attempts are valid negative stimulus, not real reads
- Lets one stimulus stream serve both **happy-path** and **negative** coverage without bookkeeping in the test

Reset-Aware Queue Scoreboard

The scoreboard is an **independent SystemVerilog queue** — it does not look inside the DUT.

```
// async_fifo_scoreboard.sv — essential pieces (verbatim signatures)
transaction_write tw[$];                    // independent expected-data queue

function void write_port_a(transaction_write txw);     // analysis_imp for write
    tw.push_back(txw);                                  // accepted write → expected
    write_count++;
endfunction

function void write_port_b(transaction_read txr);      // analysis_imp for read
    transaction_write popped;
    if (tw.size() > 0) begin
        popped = tw.pop_front();
        if (txr.rData === popped.wData)
            `uvm_info("ASYNC_FIFO_SCOREBOARD",
                $sformatf("PASSED Expected Data: %0h --- DUT Read Data: %0h",
                          popped.wData, txr.rData), UVM_HIGH)
        else
            `uvm_error("ASYNC_FIFO_SCOREBOARD",
                $sformatf("ERROR Expected Data: %0h Does not match DUT Read Data: %0h",
                          popped.wData, txr.rData))
    end else
        `uvm_error("ASYNC_FIFO_SCOREBOARD", "Read observed with empty expected queue")
endfunction
```

The **reset flush** lives inside `run_phase` (see slide 23) — the default run uses `+define+RESET_SEQUENCE` (periodic resets); without an explicit flush, the queue desyncs from the DUT after the first reset.

Coverage Strategy — DUT-Scoped Only

Coverage is **deliberately scoped to the design under test** — the testbench is not instrumented.

```
# Post_M5/UVM/run.do (verbatim line)
vopt tb_top -o top_optimized +acc +cover=sbfec+asynchronous_fifo(rtl).
```

Flag letter	Meaning
`s`	Statement coverage
`b`	Branch coverage
`f`	Finite-state-machine state coverage
`e`	Expression coverage
`c`	Condition coverage

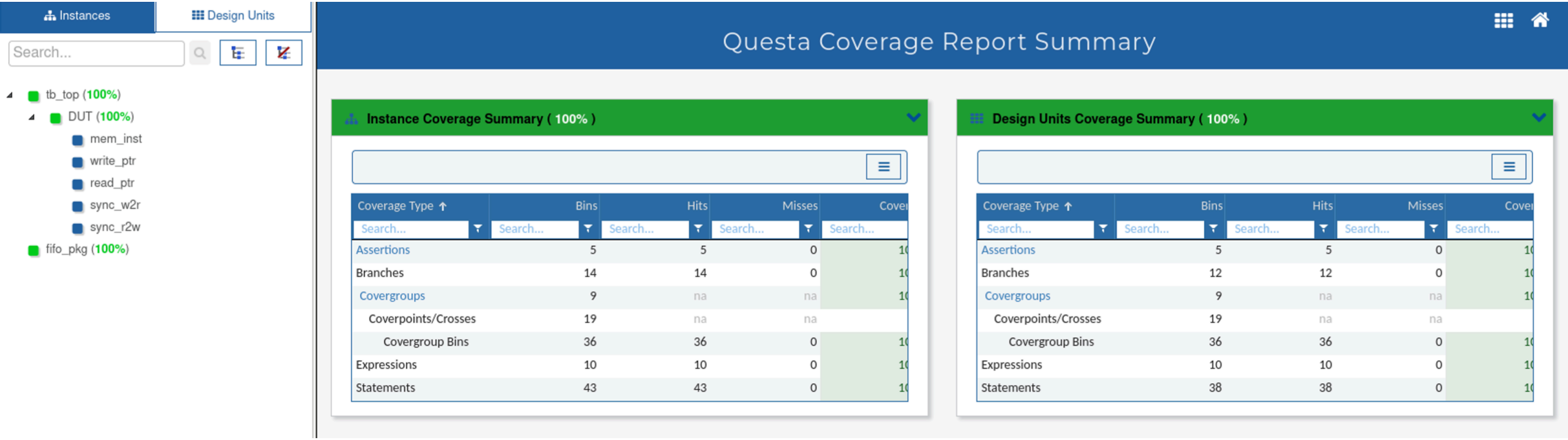
The trailing `+asynchronous_fifo(rtl).` clause says **only this instance** — TB code is excluded.

> **Why:** instrumenting the testbench would inflate the reported number with TB-internal statements. The number that matters is "how much of the DUT did the verification environment actually exercise."

Covergroup Inventory — `async_fifo_coverage.sv`

Covergroup	Bins	Purpose
`cg_fifo`	`winc`, `rinc`, `wFull`, `rEmpty`	Basic signal toggle
`cg_half_full_empty`	`wHalfFull`, `rHalfEmpty`	Mid-occupancy thresholds
`cg_data_integrity`	`wData` in 4 quartiles	Full data-range exercise
`cg_data_patterns`	`0x00`, `0xFF`, `0x55`, `0xAA` on both `wData` & `rData`	Walking-pattern stress
`cg_burst_ops`	`winc=1`, `rinc=1`	Pure-write & pure-read bursts
`cg_idle_cycles`	`winc=0 iff !winc`, `rinc=0 iff !rinc`	Quiet-period sampling
`cg_high_freq`	`wclk=1`, `rclk=1`	Clock-high coincidence
`cg_abrupt_change`	enable transitions	Sudden producer/consumer rate change
`cg_throughput`	enable cross-coverage	Sustained simultaneous activity

`cg_reset` is additionally declared in the source for completeness; the canonical farm run reports **9 covergroup types** at 100%.


Instances
Design Units
Search...
tb_top (100%)
DUT (100%)
mem_inst
write_ptr
read_ptr
sync_w2r
sync_r2w
fifo_pkg (100%)
Questa Coverage Report Summary
Instance Coverage Summary (100%)
Coverage Type
Bins
Hits
Misses
Assertions 5 5 0
Branches 14 14 0
Covergroups 9 na na
Coverpoints/Crosses 19 na na
Covergroup Bins 36 36 0
Expressions 10 10 0
Statements 43 43 0
Design Units Coverage Summary (100%)
Assertions 5 5 0
Branches 12 12 0
Covergroups 9 na na
Coverpoints/Crosses 19 na na
Covergroup Bins 36 36 0
Expressions 10 10 0
Statements 38 38 0

Bug-Injection Framework — Four `ifdef`-Gated Defects

`Post_M5/UVM/async_fifo.sv` ships with **four real defects** behind compile-time macros. They are commented in `run.do` by default; uncommenting any one makes the verification environment **fail loudly**.

Macro	Injected defect	Where	Verification purpose
`WDATA_CORRUPTION_BUG`	XORs write data with `8'hFF` before storing	`fifo_memory`	Scoreboard data-integrity
`SYNC_BUG`	Removes the second synchronizer flop (single-flop sync)	`sync`	CDC robustness
`RPTR_BUG`	Drops Gray encoding on the read pointer	`read_pointer`	Pointer encoding
`WPTR_FULLFLAG_BUG`	Naïve equality compare instead of wrap-aware Gray	`write_pointer`	Lap-parity full detection

```
# run.do — flip exactly one comment to fail the environment on demand
# vlog -source -lint +define+WDATA_CORRUPTION_BUG async_fifo.sv
# vlog -source -lint +define+SYNC_BUG             async_fifo.sv
# vlog -source -lint +define+RPTR_BUG             async_fifo.sv
# vlog -source -lint +define+WPTR_FULLFLAG_BUG    async_fifo.sv
```

A passing verification environment is necessary; **an environment that also fails for known defects is sufficient.**

Bug-Injection Evidence — `WDATA_CORRUPTION_BUG`

Enabling `WDATA_CORRUPTION_BUG` (XOR write data with `0xFF`) produces immediate, repeated scoreboard errors:

```
# Post_M5/docs/transcript_datacorruptionbug.txt (verbatim; identifier prefix elided)
UVM_ERROR async_fifo_scoreboard.sv(65) @ 260:
  [ASYNC_FIFO_SCOREBOARD] ERROR Expected Data: 2 Does not match DUT Read Data: fd
UVM_ERROR async_fifo_scoreboard.sv(65) @ 360:
  [ASYNC_FIFO_SCOREBOARD] ERROR Expected Data: d2 Does not match DUT Read Data: 2d
UVM_ERROR async_fifo_scoreboard.sv(65) @ 420:
  [ASYNC_FIFO_SCOREBOARD] ERROR Expected Data: ba Does not match DUT Read Data: 45
UVM_ERROR async_fifo_scoreboard.sv(65) @ 440:
  [ASYNC_FIFO_SCOREBOARD] ERROR Expected Data: b Does not match DUT Read Data: f4
```

Verify by hand: `0x02 ^ 0xFD = 0xFF`, `0xD2 ^ 0x2D = 0xFF`, `0xBA ^ 0x45 = 0xFF`, `0x0B ^ 0xF4 = 0xFF`. Every mismatch's xor is exactly the injected `0xFF` constant — the scoreboard is catching the bug at the precise xor relationship the code introduces, not by accident.

Milestone Progression — Five Snapshots, One Repository

The repo is organized as **a sequence of frozen verification-maturity snapshots**, not a layered codebase.

Stage	Directory	What this milestone added
M1	`M1/ConventionalTB/`	Procedural SV testbench; queue-based checking; first `$finish`-clean run
M2	`M2/CLASS/`	Class-based generator/driver/monitor/scoreboard; pre-UVM modularization
M3	`M3/CLASS/`	Added functional coverage; **uncovered the one-cycle-late half-flag bug**
M4	`M4/UVM/`	UVM agents, sequencers, sequences; analysis-port-based scoreboard
M5	`M5/UVM/`	UVM assertions, bug-injection hooks, refined run flow
Post_M5	`Post_M5/UVM/` ★	Canonical: race-free drivers, reset-aware scoreboard, farm evidence, 100% coverage

★ Canonical. Earlier milestones are preserved for grading provenance and to show *how* the methodology evolved — they are not maintained.

Engineering Insight #1 — The One-Cycle-Late Half Flag

Symptom (M3): the directed test asserted that `wHalfFull` should go high when the 32nd write was accepted. Waveform showed it asserting on the **33rd** write — one rclk too late.

Root cause: the original RTL drove half-flags off **current** `binary_wptr`. By the time the next clock edge updates `binary_wptr` to 32, the threshold transaction has already been accepted.

Wrong fix (rejected): "just shift the threshold to 31." This would make the threshold-relative semantics wrong for a parameterized depth — fragile.

Correct fix (Post_M5): drive the flag off `binary_wptr_next` (already computed combinationally for the local increment) compared against the synchronized remote pointer. The assertion edge now coincides with the accepted transfer.

> **Takeaway:** "flag timing off-by-one" is rarely a counter bug — it's a *combinational vs. registered* signal choice. Always ask which side of the flip-flop the flag wants to live on.

Engineering Insight #2 — Reset-Flush Discipline

Tempting shortcut: "disable reset stimulus so the scoreboard logic is simpler."

Why rejected: production hardware can be reset at any time. The FIFO must come back to empty cleanly and the scoreboard must stay aligned with the DUT through it — hiding reset from stimulus also hides any reset-induced bug.

What was done instead: UVM classes can't host `always` blocks, so the flush lives inside `run_phase` as a `forever` loop with an event control — the idiomatic equivalent:

```
// async_fifo_scoreboard.sv — verbatim run_phase
task run_phase(uvm_phase phase);
    super.run_phase(phase);
    forever begin
        @(negedge vif.wrst or negedge vif.rrst);
        if (tw.size() != 0)
            `uvm_info("SCOREBOARD",
                $sformatf("Reset flushed %0d queued expected writes", tw.size()), UVM_LOW)
        tw.delete();
        reset_count++;
    end
endtask
```

The farm transcript reports `Reset flushes: 2` and `Residual expected_q: 0` — the flushes fired and the queue was empty at end-of-test.

> **Takeaway:** when a feature is "hard to verify," verifying it is exactly the point — work around it and the bug ships.

Tooling & Workflow

Tool	Role
Siemens Questa 2021.3_1	Primary simulator (`vlog`, `vopt`, `vsim`, `vcover`)
UVM (bundled with Questa)	Verification methodology
PSU ECE farm (`mo.ece.pdx.edu`)	Headless farm runs for evidence capture
`~/claude-runs/<UTC-stamp>_<slug>/`	Per-run isolation directory on the farm — every agent-driven run is auditable
`make_artifacts.py`, `make_flag_debug_waveforms.py`	VCD → CSV → SVG → PNG rendering pipeline; deck images regenerate from real VCDs

Canonical non-interactive invocation:

```
cd Post_M5/UVM
vsim -c -do "do run.do; quit -f" | tee transcript
```

The explicit `quit -f` is mandatory — `run.do` sets `NoQuitOnFinish 1`, so `$finish` halts the simulation but leaves `vsim` at a Tcl prompt forever otherwise.

Final Verdict — `Post_M5/UVM/transcript_farm.txt`

```
Writes observed       : 659
Reads  observed       : 596
Reset flushes         : 2
Residual expected_q   : 0
Mismatches / errors   : 0
Verdict: *** PASSED ***

--- UVM Report Summary ---
** Report counts by severity
UVM_INFO    :   29
UVM_WARNING :    0
UVM_ERROR   :    0
UVM_FATAL   :    0
```

```
DUT filtered instance coverage: 100.00%
TOTAL COVERGROUP COVERAGE:     100.00%
Covergroup types:              9
```

- Warning-clean compile, zero errors, zero fatals — the simulation reached `$finish` on its own
- Reset stimulus exercised twice mid-run with zero residual queue entries
- 100% on both **code coverage** (statement/branch/expression/condition/FSM) and **functional coverage** (covergroups)

Summary — Skills Demonstrated

SystemVerilog & RTL — parameterized modules, generate blocks, binary-to-Gray conversion, `always_ff` reset discipline, wrap-aware comparators, dual-port memory.

UVM — `uvm_component` hierarchy, agents with sequencer/driver/monitor, `uvm_config_db` virtual interface plumbing, analysis ports/FIFOs, factory overrides, `uvm_test_top` selection via `+define+`.

CDC engineering — two-flop synchronizer placement (destination-clocked), Gray pointer encoding, lap-parity full/empty disambiguation, next-pointer half-flag timing.

Verification methodology — independent reference model (queue-based scoreboard), accepted-transfer monitoring, race-free driver scheduling, reset-aware checking, DUT-scoped coverage.

Evidence discipline — farm-reproducible runs, manifest files, parsed waveform CSVs, rendered PNG artifacts in the repo, four `ifdef`-gated bugs that the environment provably catches.

Tooling — Questa flow scripting, coverage merging across runs, headless SSH-based farm orchestration, VCD-to-PNG rendering for review without a waveform GUI.

Questions?

Deep-dive topics ready to discuss:

- The exact Gray-code wrap-aware full detection (`WPTR_FULLFLAG_BUG`)
- How the synchronizer MTBF changes when the second flop is removed (`SYNC_BUG`)
- Reset-flush event ordering and why two `always @(negedge ...)` blocks suffice
- DUT-scoped coverage filtering — why we exclude TB statements
- Farm-side artifact rendering pipeline (`make_artifacts.py`)
- Trade-offs between procedural (M1), class-based (M2/M3), and UVM (M4/M5/Post_M5) testbench architectures

Sasha Katne — github.com/sashakatne/Team3_AsyncFIFO_S24_ECE593